

Langage Go

Un langage connu car accessible sur github, puissant, accessible et résout un problème.

Langage go

compilé

syntaxe du c++

proche du C#

vs

Python

Syntaxe facile

peut presque tout faire

interprété avec couche intermédiaire avec couche CPU

point négatif du go :

complexité

rigidité

recompilation à chaque modif

risque de fuite de données

Langage Objet

Java

C / C++ / C#

différence entre C et Java : Garbage collector (meilleure gestion de données non utilisées)

La création de go

Objectifs d'un nouveau langage

- bas niveau
- web/mobile
- performance

Le langage orienté objet :

- héritage de classe
 - objet enfant récupère des infos/propriétés du parents
- composition

- Peut être construit avec des infos/données d'une autres classes

héritage et composition sont différents.

Go

Avantage

Simplicité (C est trop compliqué sur la mémoire et la syntaxe ect)

- à écrire
- à lire

Rapidité (Performance)

- exécution
- compilation

Accès concurrentielles

2 threads veulent accéder à la même ressource.

- Problème mémoire (RAM)
- Données corrompues
- Données bloquées (deadlock)

Pour corriger/simplifier le multi-threading = Go Routines

Go permet le pragmatisme

Sauf qu'on impose une seule façon de faire (idiomatique) permet de simplifier la lecture du code et la compréhension du code

- Permet la stabilité dans le temps
- Permet la compatibilité

Compromis entre performance, rapidité de dev, la simplicité de dev. performance largement supérieur que Javascript et Python.

L'objectif est de rendre le code simple à écrire et à lire

Grosse restriction dans la méthode d'écriture. comme les "{}". Dans tout les .go il va y avoir un `import fmt` pour être rigide dans l'écriture. Rigidité pour simplification.

MAJ régulière donc beaucoup de restriction dans la réalisation de code.

Les contraintes permettent de s'améliorer.

Nombre de mots syntaxiques dans

C : 45 mots

5 millions de développeurs Go

Docker, Kubernetes, Terraform, Prometheus sont écrits en Go

Go

→ Outils CLI

→ micro service api rest

→ Agent devops, deamon system

→ infra cloud (IAS)

→ Kotling (plutôt utiliser Java)

Héritage du C : Il y a un package main à avoir.

```
package main

import (
    "fmt"
    "math"
    "time"
)

func main() {
    fmt.Printf("64 puissance 8 fait : %g\n", math.Pow(64, 8))
    // date with format yyyy-mm-dd hh:mm:ss.ms
    date := time.Now()
    fmt.Printf("Nous sommes le %d %d %d et il est %d:%d\n", date.Day(), date.Month(), date.Year(), date.Hour(),
date.Minute())
    // birthdate with format yyyy-mm-dd hh:mm:ss.ms
    birthDate := time.Date(2003, 2, 28, 0, 0, 0, 0, time.UTC)
    fmt.Printf("Je suis né le %d %d %d\n", birthDate.Day(), birthDate.Month(), birthDate.Year())

    // Différence entre les deux dates
    diff := date.Sub(birthDate)
    // jours
    totalDays := int(diff.Hours() / 24)
    // Années
    years := totalDays / 365
    // Jours restants
    days := totalDays % 365

    fmt.Printf("Et j'ai %d ans et %d jours\n", years, days)
}
```

```
$ go run .
64 puissance 8 fait : 2.81474976710656e+14
Nous sommes le 26 5 2026 et il est 17:50
Je suis né le 28 2 2003
Et j'ai 23 ans et 93 jours
```

Recherche :

- go run

Créer des multi threads pour lancer directement dans le terminal le script en go.

- go build (avec ses options)

Créer un exécutable puis l'exécute. Il peut y avoir des options dans la structures suivante :

```
go build [-o output] [build flags] [packages]
```

Dans le cas d'aucune options

[-o output] : contient le nom du fichier que l'on souhaite donner à l'exécutable

[build flags] : peut contenir des options pour compléter le build

[packages] : spécifier le package que l'on souhaite compiler (. pour tout le projet)

- go build -ldflags "-s -w" hello.go

`-s` : supprime la table des symboles

`-w` : supprime les informations de debug

Les types en Go

Les entiers typé

int 8

int 16

int 32

Les entiers non typé (toujours positif)

uint8

uint16

uint32

uint64

Les floats

float32

float64

Boolean

var bool1 = true

String

var text = "test de text"

→ Concaténation possible

→ indexable (recherche, optimisation, ...)

Spéciaux (Alias)

byte = uint8 (principalement dans la manip de fichier)

rune = int32 (caractère unicode)

NULL (classique)= Nil (dans les listes)

Affichage

Printf %d = entier

%.2f = flottant

%s = Chaîne de caractère

%T = le type de la données

%v = affiche la valeur

Il n'y a pas de conversion native

Lors de l'initialisation il n'y a pas de valeur null il y a des valeurs par défauts.

iota : définit des constantes incrémentales en fonction du nombre de constantes définies précédemment dans un même bloc const () comme

```
const (  
    Sunday    = iota // 0  
    Monday    = iota // 1  
    Tuesday   = iota // 2  
    Wednesday = iota // 3  
    Thursday  = iota // 4  
    Friday    = iota // 5  
    Saturday  = iota // 6  
)
```

ça permet d'incrémenter des variables en fonction de leur déclaration

on peut custom l'incrémentation avec des formules

```
const (  
    Sunday    = 1 << (10 * (iota + 1)) //1024  
    Monday    //1048576  
    Tuesday   //...  
    Wednesday //...  
    Thursday  //...  
    Friday    //...  
    Saturday  //...  
)
```

Séance 2 :

Un slice est un tableau dont la taille est modifiable

Var non nécessaire mais il faut l'utiliser

```
Var age = 30
```

ou

```
age := 30
```

:= n'est utilisable seulement dans les fonctions

possible de définir plusieurs variables

a, b := 10, 20

possible d'intervertir deux variables

x, y = y, x

liens vers répo github : <https://github.com/Valentin-Gelly/langage-go>

TP1 :

 **EXERCICE NOTÉ 1 — Calculateur d'IMC et de catégorie** [15 min]

Contexte : L'IMC (Indice de Masse Corporelle) est utilisé en médecine. C'est un premier exercice pratique mêlant types, variables et constantes.

1. Déclarez deux variables float64 : poids (ex. 70.5 kg) et taille (ex. 1.75 m)
2. Créez 4 constantes : IMC Maigre=18.5, IMC Normal=25.0, IMC Surpoids=30.0
3. Calculez l'IMC = poids / (taille * taille)
4. Affichez l'IMC avec 2 décimales (fmt.Printf avec %.2f)
5. Affichez la catégorie : Maigre / Normal / Surpoids / Obésité
6. BONUS : déclarez une constante Nom avec votre prénom et personnalisez l'affichage

Rendu : fichier imc.go sur GitHub

Indices : `fmt.Printf("IMC : %.2f\n", imc) | if/else if/else | const (A = 1.0; B = 2.0)`

Dans le langage go il n'y a pas de surcharge de fonction.

Il y a un problème de compréhension quand on crée des méthodes qui existent déjà.

_ permet d'ignorer une valeur (dans un buffer par exemple)

définition du type des fonctions entre (). Pareil un naked return retourne des valeurs par défaut.

Les erreurs sont des valeurs à gérer.

```
res, err = addition(a,b)
if err != nil { ... }
```

méthode variadique

```
func afficherNombres(nombres ...int) {
    for _, n := range nombres {
        fmt.Println(n)
    }
}
```

closure :

c'est une fonction renvoie une fonction

```
func creerMultiplieur(facteur int) func(int) int {
    return func(x int) int {
        return x * facteur // facteur est mémorisé
    }
}
```

c'est une fonction qui se souvient de l'environnement dans lequel elle a été créée

boucle for

il n'y a que la boucle for

while =>

for n < 100 { ... }

for infini - break pour sortir

for range => utilisation d'un slice ou d'un tableau



EXERCICE NOTÉ 2 — Calculatrice avec closures et validation [20 min]

Contexte : Une calculatrice CLI qui démontre les fonctions, retours multiples et closures.

1. Créez une fonction `operer(a, b float64, op string) (float64, error)`
qui supporte '+', '-', '*', '/' — retourne une erreur pour / par 0 ou op inconnue
2. Créez une fonction `creerOperation(op string) func(float64, float64) float64`
qui retourne une closure pour chaque opération
3. Dans `main()`, faites une boucle 'while true' qui :
 - Lit deux nombres et une opération depuis l'entrée standard (`fmt.Scan`)
 - Affiche le résultat ou l'erreur
 - Sort si l'opération est 'quit'
4. Testez avec : `10 5 + → 15 | 8 0 / → erreur | 3 4 * → 12`

Rendu : fichier `calculatrice.go` sur GitHub

Indices : `switch op { case '+': ... } | fmt.Scan(&a, &b, &op) | for { if op == 'quit' { break } }`

fallthrough :

Permet de déclencher les cas d'un switch case qui sont en dessous du cas validé.

```
v := 42
switch v {
case 100:
    fmt.Println(100)
    fallthrough
case 42:
    fmt.Println(42)
    fallthrough
case 1:
    fmt.Println(1)
    fallthrough
default:
    fmt.Println("default")
}
// Output:
// 42
// 1
// default
```

Maps

dictionnaire clés valeur

Copy de slice

copy les valeurs dans un autre slice

```
arr := []int{1, 2, 3}
tmp := make([]int, len(arr))
copy(tmp, arr)
fmt.Println(tmp)
fmt.Println(arr)
```

Struct

notion de classe

```
type XXXX struct {
    ...
}
```

3 façon d'initialiser les struc

1. positionnel
2. Nommée

```
p2 := XXX {
    propriete1: valeur1,
    propriete2: valeur2
}
```

3. Valeu zéro puis affectation

```
var p3 XXX
pe.propriete1= valeur1
pe.propriete2= valeur2
```

Les tags

Il est possible de mettre des tags dans un structs

```
type XXXX struct {
    ID      int      'json:id'           //sérialisé comme "id"
    Prenom  string  'json: prenom'
    Prenom  string  'json: prenom, omitempty' // omis si vide ("" )
    Interne string  'json:"-"' // Jamais sérialisé
}
```

Les pointeurs

Ils permettent de modifier les valeurs directement qui est en mémoire pour éviter la duplication de données.

On parle de value receiver pour afficher les données et on parle de pointer receiver quand on modifie les valeurs.

Exercice :

EXERCICE NOTÉ 3 — Système de contacts : Personne, Employé, Étudiant [20 min]

Contexte : Modéliser un annuaire d'entreprise avec différents types de contacts.

1. Créez une struct `Personne` { `Prenom` string; `Age` int; `Email` string } avec la méthode `NomComplet()` string et `Presentation()` string
2. Créez `Adresse` { `Rue`, `Ville`, `CodePostal` string } avec `Format()` string
3. Créez `Employe` en embeddant `Personne` et `Adresse` + champs `Poste` string, `Salaire` float64
Ajoutez la méthode `FicheEmploye()` string qui affiche toutes les infos
Ajoutez `AugmenterSalaire(pct float64)` qui modifie le salaire
4. Créez `Etudiant` en embeddant `Personne` + champs `Promo` string, `Moyenne` float64
Ajoutez la méthode `MentionObtenue()` string (TB/B/AB/P selon moyenne)
5. Dans `main()`, créez 2 employés et 2 étudiants, affichez leurs fiches

Rendu : `contacts.go` sur GitHub

Indices : (e `*Employe`) pour modifier le salaire | `switch { case moy >= 16: ... } | fmt.Sprintf` pour construire des chaînes

!!! Il faut mettre la première lettre en minuscule dans le cas d'une propriété private ou maj pour public

```
// EXPORTÉ — commence par une majuscule — accessible depuis n'importe où
type Produit struct {
    Nom    string
    Prix   float64 // exporté
    stock int    // NON exporté — uniquement dans le package boutique
}
```

Defer

mot clés pour exécuter du code même si ça crash par la suite.

last in first out donc le dernier defer écrit dans la fonction est le premier à être exécuté.

TP 1

EXERCICE NOTÉ TP1 — TechShop — Boutique CLI [45 min autonome + rendu GitHub]

Contexte : Vous êtes développeur chez TechShop, une boutique en ligne de matériel informatique. Vous devez créer un gestionnaire de catalogue en ligne de commande.

=== DONNÉES ===

- Struct Produit : ID int, Nom string, Marque string, Prix float64, Stock int, Catégorie string (actif bool)
- Struct Catalogue avec une slice de Produits

=== FONCTIONNALITÉS (méthodes sur Catalogue) ===

- AjouterProduit(p Produit) error — erreur si ID dupliqué
- TrouverParID(id int) (Produit, error)
- TrouverParCatégorie(cat string) []Produit
- AppliquerRéduction(catégorie string, pct float64) int — retourne nb de produits modifiés
- Vendre(id int, qte int) error — réduit le stock, erreur si insuffisant
- Rapport() string — résumé : nb produits, valeur totale du stock

=== MAIN ===

- Menu CLI interactif : [1] Ajouter [2] Chercher [3] Soldes [4] Vendre [5] Rapport [0] Quitter
- Pré-remplissez 5 produits high-tech réalistes (iPhone, MacBook, etc.)
- Gérez toutes les erreurs proprement

=== RENDU ===

- Fichier : tp1_techshop.go (tout dans un seul fichier)
- Commande : git init + git add + git commit + git push sur votre GitHub
- URL du repo à envoyer dans le chat Teams

=== CRITÈRES D'ÉVALUATION ===

- /5 : Structs + méthodes correctes (value vs pointer receiver)
- /5 : Gestion des erreurs complète
- /5 : Menu CLI fonctionnel
- /5 : Code propre, noms explicites, package main correct

Indices : `fmt.Scan(&choix) | for i, p := range c.produits { if p.ID == id { ... } } | strings.EqualFold` pour comparaison insensible à la casse